Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Scrabble – Java

Rapport final

CODOGNAN Thomas, RAZE Erwan, KAHLOUL Aïssa, BLOTHIAUX Yanis, BETTCHER Léonard

10 avril 2026

1 Explication détaillée du sujet

1.1 Historique du sujet

En 1931, l'architecte au chômage Alfred Mosher Butts décide de créer un jeu combinant chance et réflexion. En analysant minutieusement la fréquence des lettres à la une du *New York Times*, il élabore un système de valeurs spécifiques pour chaque jeton, méthode qui reste inchangée dans les éditions modernes [1]. Initialement baptisé *Lexiko*, puis *Criss-Cross Words*, le jeu essuie les refus de nombreux fabricants avant qu'un homme d'affaires, James Brunot, ne rachète les droits en 1948 pour lui donner son nom définitif : Scrabble [2].

Le succès mondial du jeu doit beaucoup à un coup de chance marketing. Dans les années 1950, Jack Straus, président du grand magasin Macy's à New York, découvre le jeu pendant ses vacances et s'étonne de ne pas le trouver dans ses rayons. Il passe une commande massive, propulsant le Scrabble au rang de phénomène culturel [2]. Aujourd'hui, avec plus de 150 millions d'exemplaires vendus dans 121 pays et des compétitions internationales organisées par la World English-Language Scrabble Players Association (WESPA) et la Fédération Internationale de Scrabble Francophone (FISF), il est devenu le jeu de lettres le plus populaire de la planète [3].

Dès les années 1980 et 1990, les premiers logiciels de jeu contre l'ordinateur font leur apparition. L'explosion des plateformes en ligne, notamment l'Internet Scrabble Club (ISC), puis des applications mobiles comme *Scrabble Go* ont élargi considérablement la communauté de joueurs. Au-delà du divertissement, l'informatique a révolutionné la compétition de haut niveau grâce à des outils d'analyse stratégique comme Quackle [4], devenu la référence mondiale pour l'entraînement des joueurs experts. Aujourd'hui, de nombreux projets open source et applications multiplateformes témoignent de la vitalité de l'écosystème logiciel autour du Scrabble.

1.2 Nature et enjeux du jeu

Le Scrabble est un jeu de lettres à information imparfaite (les tirages adverses sont inconnus) et stochastique (soumis à l'aléa de la pioche). L'objectif est de maximiser son score sur une grille standard de 15×15 cases en y plaçant des mots extraits d'un dictionnaire de référence (ODS en français, TWL/SOWPODS en anglais).

Le jeu repose sur trois piliers fondamentaux :

1. **Logique spatiale** Tout mot posé doit respecter deux contraintes simultanées : la connexité (chaque nouveau mot doit s'appuyer sur au moins une lettre déjà présente sur le plateau, et toutes les lettres posées en un même tour doivent être alignées et contigües) et l'orthogonalité (toute collision avec des lettres adjacentes doit former un mot valide dans le dictionnaire de référence).

   Exemple : si le mot FEU est posé horizontalement, un joueur peut placer la lettre R au-dessus du E pour former RE verticalement, à condition de poser simultanément un mot horizontal valide passant par ce R, comme RATS.

2. **Logique mathématique** Le score d'un coup se calcule en additionnant la valeur de chaque lettre posée, en appliquant d'abord les multiplicateurs de lettres (case lettre compte double/triple), puis les multiplicateurs de mots (case mot compte double/triple). Un joueur qui utilise ses 7 lettres en un seul coup remporte un bonus de 50 points, appelé Scrabble.

   Exemple : au premier tour, un joueur pose AXE avec le X sur la case centrale H8 (case mot $\times 2$). La valeur brute est A(1) + X(8) + E(1) = 10, doublée à 20 points. Si la case centrale était une case lettre $\times 2$ sous le X, on calculerait A(1) + X(16) + E(1) = 18, puis on appliquerait le multiplicateur de mot.

3. **Logique linguistique** Chaque mot posé, y compris les mots formés par collision, doit appartenir au dictionnaire de référence choisi pour la partie. Tout mot invalide entraîne l'annulation du coup et, selon les règles, une pénalité de score.

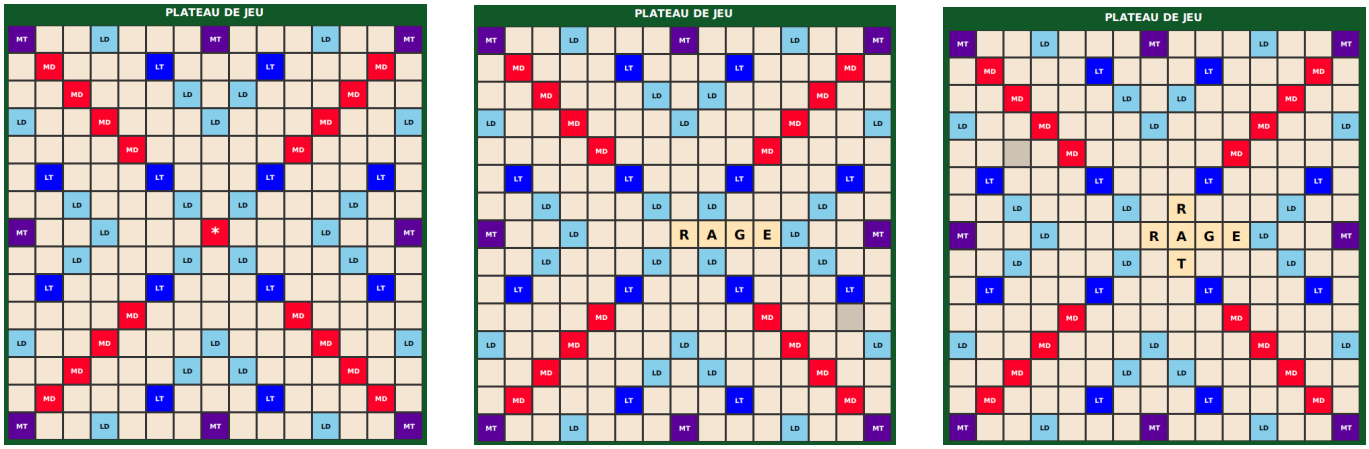


FIGURE 1 – Trois captures des premiers coups d'une partie, illustrant la mise en place progressive du plateau et l'application des règles de pose.

Déroulement d'une partie La partie débute obligatoirement par un mot d'au moins deux lettres couvrant la case centrale (H8), dont le score est automatiquement doublé. À chaque tour, le joueur peut : poser un mot, échanger tout ou partie de ses jetons contre des lettres de la pioche (si le sac contient au moins 7 lettres), ou passer son tour. La partie s'achève lorsque le sac est vide et qu'un joueur a épuisé son chevalet, ou après trois tours consécutifs sans score. Les lettres restant sur les chevalets sont déduites du score de chaque joueur, et leur valeur totale est ajoutée au score du joueur ayant vidé son chevalet en premier.

Exemple de fin de partie : le joueur A termine avec 300 points. Le joueur B possède encore un W (10 pts) et un X (10 pts) : son score final est 280 pts, et le score de A monte à 320 pts.

1.3 Complexité du jeu et défis algorithmiques

Il est essentiel de distinguer la complexité intrinsèque du jeu de celle des algorithmes qui cherchent à le résoudre.

Espace des états Le nombre de positions légales sur un plateau 15×15 est estimé à environ 10^{30}. Si ce chiffre reste inférieur à celui des échecs ($\sim 10^{43}$), l'incertitude propre au Scrabble, due aux tirages cachés de l'adversaire et à l'aléa de la pioche, en fait un problème fondamentalement différent : il relève de la théorie des jeux à information imparfaite, contrairement aux échecs ou aux dames.

Explosion combinatoire à chaque coup Même pour un seul tour, la génération de tous les coups légaux est un problème non trivial. Pour un chevalet de 7 lettres et un dictionnaire de type ODS ($\sim 380\,000$ mots), il faut explorer toutes les combinaisons de lettres (sous-ensembles du chevalet), toutes les positions et orientations possibles sur la grille, et vérifier simultanément la validité de tous les mots formés par collision. Le nombre de placements théoriques au premier tour est déjà de l'ordre de plusieurs millions [5].

La qualité du reliquat Un des défis stratégiques majeurs, et algorithmiques, est que maximiser le score immédiat n'est pas toujours optimal. Un bon moteur de jeu doit évaluer la qualité du reliquat (les lettres conservées après le coup), car des lettres polyvalentes comme les voyelles courantes ou le S offrent de meilleures perspectives pour les tours suivants. À l'inverse, conserver des lettres difficiles comme le W ou le K pénalise les coups futurs.

La gestion des cases bonus Un autre défi crucial est l'obstruction : ouvrir une case bonus (mot $\times 3$ ou lettre $\times 3$ en bord de grille) pour l'adversaire peut lui offrir un avantage considérable. L'IA doit donc anticiper non seulement son propre score, mais aussi les opportunités qu'elle laisse à son adversaire.

Conclusion sur la difficulté Ces contraintes font du Scrabble un problème d'optimisation combinatoire sous incertitude, à la frontière entre la recherche arborescente, la linguistique computationnelle et la théorie des jeux stochastiques.

1.4 Existant algorithmique

L'état de l'art repose sur des travaux académiques fondateurs et des logiciels éprouvés.

Structures de données pour le dictionnaire La génération rapide de coups légaux repose sur des automates finis représentant le dictionnaire :

- le DAWG (*Directed Acyclic Word Graph*), proposé par Appel & Jacobson (1988) [5], compresse le dictionnaire en un graphe orienté acyclique permettant une vérification en $O(n)$ de la validité d'un mot ;
- le GADDAG, introduit par Steven Gordon (1994) [6], est une extension permettant de générer directement tous les mots pouvant s'ancrer sur une lettre donnée du plateau, accélérant considérablement la recherche des coups légaux.

Logiciels de référence Quackle [4] est le standard académique et competitif pour l'analyse stratégique. Il utilise des simulations de type Monte-Carlo pour estimer l'espérance de gain de chaque coup sur plusieurs tours à venir, en simulant des milliers de parties possibles.

Elise et Maven sont d'autres moteurs historiques ayant contribue à l'etablissement des meilleures pratiques en IA Scrabble.

Plateformes de jeu en ligne L'ISC (*Internet Scrabble Club*) demeure la référence pour la pratique competitive, garantissant une application stricte des règles internationales. Des plateformes plus recentes comme Woogles.io proposent également des outils d'analyse intégrés.

Open source De nombreux projets open source (notamment en Python, Java et C++) sont disponibles sur des plateformes comme GitHub, rendant accessible l'implementation d'un moteur de jeu à partir des algorithmes decrits dans la litterature.

2 Algorithmes et Structure de Données

Le cœur décisionnel de notre intelligence artificielle repose sur des algorithmes de recherche arborescente. Le défi majeur du Scrabble étant l'information imparfaite (le chevalet adverse est caché), nous avons dû adapter ces structures de données classiques à des environnements probabilistes.

2.1 L'Algorithme Minimax

Historiquement conçu pour des jeux à information parfaite et à somme nulle comme les Échecs, l'algorithme Minimax modélise l'arbre des coups possibles en alternant un tour où l'IA cherche à maximiser son score, et un tour où elle simule un adversaire cherchant à minimiser l'avantage de l'IA.

Pour adapter cet algorithme au Scrabble, notre implémentation déduit d'abord les tuiles restantes, à savoir toutes les tuiles moins celles présentes sur le plateau et dans le chevalet de l'IA. Ensuite, le solveur utilise une méthode de Monte-Carlo[7] : il simule l'adversaire en tirant aléatoirement 200 chevalets virtuels de 7 lettres parmi ces tuiles restantes. Le Minimax évalue alors le pire scénario possible.

Voici comment cette logique pessimiste se traduit dans l'implémentation de notre `MinimaxSolver` :

```
    // Extrait de minimax() : Évaluation du pire scénario (Adversaire)
    double maxDamage = Double.NEGATIVE_INFINITY;

    for (int i = 0; i < SAMPLES_COUNT; i++) {
        // 1. Simulation d'un chevalet adverse aléatoire
        Character[] simulatedRack = drawRandomRack(unseen, 7);

        // 2. Recherche du meilleur coup possible pour l'adversaire
        double oppScore = getBestOpponentScore(board, simulatedRack, gaddag);

        // 3. Agrégation : on ne retient que le score maximum (le pire pour nous)
        if (oppScore > maxDamage) {
            maxDamage = oppScore;
        }
    }
    return maxDamage;
```

2.1.1 L'impact de l'heuristique

Au Scrabble, évaluer un coup uniquement par les points qu'il rapporte sur le plateau n'est pas le plus pertinent. L'algorithme intègre donc une fonction heuristique d'évaluation du "reliquat" (`leaveScore`) qui note la qualité des lettres conservées sur le chevalet après un coup. Cette heuristique va attribuer des bonus ou des malus en fonction de la "qualité" du chevalet : Joker (+15.0), 'S' (+8.0), Lettres difficiles (-6.0), équilibre voyelles/consonnes (+5.0), uniquement consonne/voyelle (-12.0).

2.1.2 Complexité, Coût et Performances

Dans notre implémentation de Monte-Carlo, l'algorithme ne dispose pas d'élagage Alpha-Beta. La complexité est donc strictement de $O(B \times N \times B)$ pour une anticipation de deux plis (Profondeur = 2), où B représente le facteur de branchement (nombre de coups légaux) et $N = 200$ le nombre de tirages simulés. Le coût en ressources est massif et identique pour Minimax et Expectiminimax, car tous deux parcourent la même boucle de 200 simulations par coup. Le goulot d'étranglement réside dans l'appel répété au `MoveGenerator` pour chaque scénario adverse. Sur le plan du jeu, la performance défensive du Minimax est excellente : l'IA « verrouille » le plateau pour empêcher l'adversaire de marquer. Cependant, ce pessimisme est à double tranchant : en s'attendant toujours au meilleur tirage adverse, l'IA ferme le jeu, sacrifiant parfois des opportunités offensives pour éviter un risque statistique pourtant faible.

2.2 L'Algorithme Expectiminimax

L'Expectiminimax est l'évolution naturelle du Minimax pour les jeux intégrant du hasard. Il introduit un nouveau type de nœud dans l'arbre de recherche : les "nœuds chance"[8]. Au lieu de considérer que l'adversaire jouera le pire coup absolu, l'algorithme calcule l'espérance mathématique de tous les coups possibles en fonction de leur probabilité d'occurrence.

Comme le montre notre implémentation ci-dessous, la structure de données est identique au Minimax (la même boucle de 200 itérations), mais la méthode d'agrégation change :

```
    // Extrait de expectiminimax() : Calcul de l'espérance mathématique
    double totalExpectedOpponentScore = 0.0;
    int samplesEvaluated = 0;

    for (int i = 0; i < SAMPLES_COUNT; i++) {
        Character[] simulatedRack = drawRandomRack(unseen, 7);

        // Cumul de tous les meilleurs coups adverses trouvés
        totalExpectedOpponentScore += getBestOpponentScore(board, simulatedRack, gaddag);
        samplesEvaluated++;
    }
    // Agrégation : on retourne la moyenne (le risque mathématique attendu)
    return samplesEvaluated == 0 ? 0.0 : totalExpectedOpponentScore / samplesEvaluated;
```

2.2.1 L'impact de l'heuristique

En termes simples, l'Expectiminimax ne se contente pas de calculer mathématiquement les risques sur le plateau. Pour juger de la force réelle d'un coup, l'IA additionne le risque statistique sur le plateau et la qualité de sa main évaluée par l'heuristique. L'IA cherche donc toujours l'équilibre parfait entre "limiter la casse probabiliste" et "garder de bonnes lettres pour l'avenir".

2.2.2 Complexité, coût et performances

La complexité est la même ($O(B \times N \times B)$). Bien que son coût soit strictement identique à celui de notre Minimax actuel, l'Expectiminimax à l'inconvénient théorique de ne presque pas pouvoir être optimisé par des techniques d'élagage classiques. Le style de jeu devient plus naturel et audacieux. Dans notre code, son coût temporel étant identique au Minimax Monte-Carlo, l'Expectiminimax s'avère supérieur dans la prise de ses décisions.

2.3 Comparaison des arbres de décision

Les schémas ci-dessous mettent en évidence que la différence entre nos deux algorithmes n'est pas structurelle (l'arbre exploré est identique), mais réside uniquement dans la méthode mathématique d'agrégation des nœuds de simulation, comme démontré dans nos extraits de code précédents.

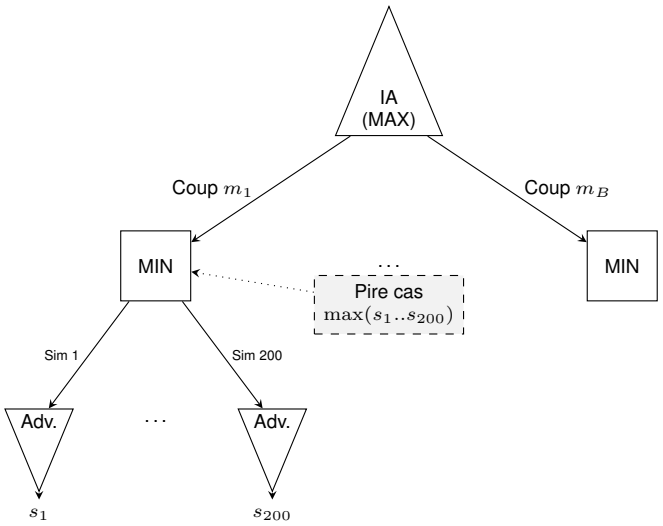


FIGURE 2 – **Agrégation Pessimiste**

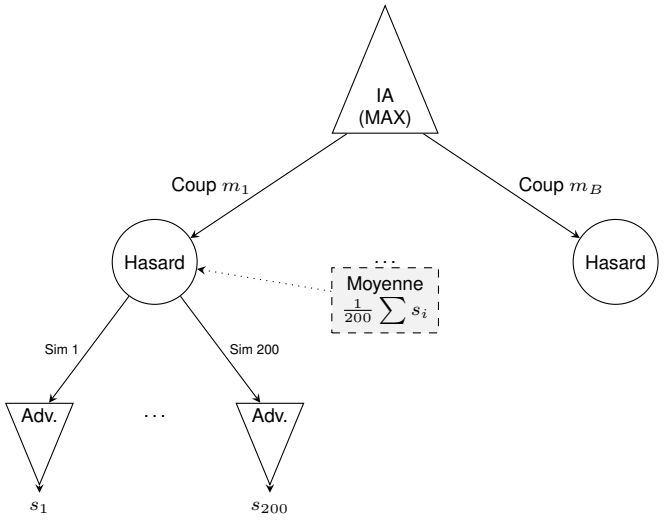


FIGURE 3 – **Agrégation Probabiliste**

FIGURE 4 – Comparaison des Arbres de Décision

2.4 Analyse des performances (Benchmark)

Afin de confronter notre étude théorique à la réalité matérielle, nous avons nous-mêmes mis en place un benchmark interne. Pour ce faire, nous avons instrumenté le code de notre solveur pour extraire des métriques physiques en conditions réelles de jeu. Le chronomètre d'interruption (`timeLimitMs`) de notre IA était fixé à 5000 millisecondes.

Le tableau ci-dessous synthétise nos résultats, basés sur des échantillons représentatifs de la prise de décision de notre moteur au cours d'une partie complète :

Algorithme	Temps	Statut	Coups IA Évalués	Simulations Adv.	Moy. Sim/Coup	Tour
Expectiminimax	5000 ms	Timeout	48 sur 84 (57%)	9 497	197	1er tour
Expectiminimax	3575 ms	Complet	33 sur 33 (100%)	6 600	200	2ème tour
Minimax	5001 ms	Timeout	24 sur 58 (41%)	4 680	195	1er tour
Minimax	5001 ms	Timeout	17 sur 41 (41%)	3 294	193	2ème tour
Minimax	2801 ms	Complet	10 sur 10 (100%)	2 000	200	3ème tour

TABLE 1 – Résultats expérimentaux durant une partie(Profondeur 2, SAMPLES_COUNT = 200)

2.4.1 Interprétation de nos résultats

L'analyse de nos métriques met en évidence trois points cruciaux concernant les limites de notre implémentation :

— **La limite du facteur de branchement :** Nos données prouvent que la capacité de l'IA à évaluer l'arbre complet dépend entièrement de la complexité du plateau à l'instant T. Lorsque le nombre de mots possibles est faible (ex : 10 ou 33 coups), le moteur termine son parcours de Profondeur 2 confortablement avant le temps limite (2.8s et 3.5s). À l'inverse, dès que les choix s'élargissent, l'IA est systématiquement bridée.

— **Le coût temporel :** Dans le pire scénario que nous avons mesuré (17 coups évalués sur 41), le *Timeout* force l'IA à jouer en ignorant totalement 59% des opportunités légales, l'empêchant potentiellement de trouver le mot optimal.

— **L'efficacité de notre coupe-circuit :** Notre moteur est capable de simuler un volume massif de plateaux adverses (jusqu'à 9 497 en 5 secondes). Lorsque le *Timeout* est déclenché, la moyenne des simulations tombe légèrement sous la barre des 200. Cela démontre que notre algorithme d'interruption fonctionne avec précision : il avorte la boucle interne de Monte-Carlo au milieu de l'évaluation pour garantir la fluidité du jeu en temps réel.

Ces mesures que nous avons relevées confirment notre théorie : bien que la Profondeur 2 soit mathématiquement atteinte, l'absence d'élagage dans notre approche de Monte-Carlo bride physiquement la rentabilité de l'arbre dès que le facteur de branchement dépasse la trentaine de mots candidats.

2.5 Le Machine Learning

Contrairement aux algorithmes de recherche arborescente qui calculent toutes les possibilités de manière exhaustive, notre approche par Machine Learning propose une méthode statistique. L'objectif de ce modèle n'est pas d'évaluer le plateau, mais d'agir comme un "générateur d'anagrammes intuitif" capable de trouver instantanément les meilleurs mots possibles à partir du chevalet de l'IA.

Architecture du Modèle

Le problème est modélisé comme une tâche de classification multiclasse. L'architecture de notre réseau de neurones (entraîné via TensorFlow et accessible via notre script `train_ml.sh`) se décompose ainsi :

- **Entrée :** Un tenseur de dimension 26 représentant le chevalet du joueur (fréquence de chaque lettre de l'alphabet).
- **Couches cachées :** Deux couches denses (Dense layers) [9]de 256 et 512 neurones utilisant la fonction d'activation ReLU.
- **Sortie :** Une distribution Softmax sur l'ensemble du dictionnaire (lexique), où chaque neurone de sortie représente la probabilité qu'un mot spécifique soit formable.

Implémentation et Intégration

Une fois le modèle chargé en mémoire, l'IA interroge le réseau de neurones pour obtenir le "Top 100" des mots les plus probables. Cependant, le modèle étant "aveugle" à la géométrie du plateau, ses prédictions doivent être croisées avec la réalité du jeu.

Voici comment notre classe `AiPlayer` gère cette hybridation entre prédiction statistique et validation algorithmique :

```
    // Phase 1 : Le réseau de neurones prédit les 100 meilleurs mots (O(1))
    List<String> mlPredictions = mlAgent.predictWords(rackStr, 100);

    // Phase 2 : Le moteur génère les mots légalement jouables sur le plateau
    List<PlayableWord> allLegalMoves = moveGen.getPlayableWordsList(game, gaddag);

    // Phase 3 : Validation croisée
    for (String predictedWord : mlPredictions) {
        for (PlayableWord legalMove : allLegalMoves) {
            // Si la prédiction de l'IA correspond à un coup légal, on le joue !
            if (legalMove.getWord().equals(predictedWord)) {
                bestPlay = legalMove;
                break; // Coup trouvé, on arrête la recherche
            }
        }
        if (bestPlay != null) break;
    }
```

Complexité, coût et performances

L'avantage majeur de cette approche est son temps d'inférence (la prédiction). Contrairement à la recherche arborescente, la complexité de la prédiction est constante, soit $O(1)$. Si le coût processeur en cours de partie est extrêmement faible comparé au Minimax, le coût en mémoire vive est nettement plus élevé, car le modèle pré-entraîné complet doit être maintenu en mémoire. De plus, l'entraînement préalable du modèle nécessite une phase de calcul très lourde en amont du lancement du jeu. En termes de génération de vocabulaire, le Machine Learning est un accélérateur extrêmement efficace. Néanmoins, sa faiblesse réside dans sa méconnaissance de l'état du jeu : il cherche à placer le mot le plus long ou le plus complexe, sans savoir s'il va ouvrir une case "Mot Compte Triple" à l'adversaire. C'est pourquoi notre implémentation inclut systématiquement un mécanisme de repli (*fallback*) vers l'algorithme Minimax ou Expectiminimax classique si aucune prédiction du réseau ne s'adapte au plateau actuel.

2.6 Le problème DAWG/GADDAG

La génération de coups légaux au Scrabble impose une contrainte forte : la structure doit, en temps réel, trouver tous les mots jouables à partir d'une lettre déjà posée sur le plateau, que ce soit vers la gauche, la droite, ou les deux. Les structures classiques comme les tableaux de chaînes ou les arbres préfixes naïfs (Trie) sont inadaptées à cette contrainte, soit par leur lenteur de parcours, soit par leur consommation mémoire excessive face à un dictionnaire de 180 000 mots comme ceux que nous avons utilisés.

Deux structures de données académiques répondent à ce problème : le DAWG (Directed Acyclic Word Graph), conçu pour la validation rapide de mots, et le GADDAG, une extension inventée spécifiquement pour le Scrabble par Steven Gordon en 1994 permettant une recherche bidirectionnelle depuis n'importe quelle ancre. Afin de justifier notre choix final, nous avons implémenté et benchmarké les deux structures sur notre dictionnaire.

2.7 Le DAWG — Directed Acyclic Word Graph

La structure de données DAWG représente une solution performante alliant vitesse d'exécution et économie de mémoire. Il s'agit d'un automate fini déterministe où chaque chemin partant de la racine correspond à un mot unique du dictionnaire. Sa particularité, qui la distingue d'un arbre préfixe classique, réside dans son optimisation poussée : le DAWG regroupe les mots partageant les mêmes préfixes, mais aussi les mêmes suffixes.
Par exemple, les mots CHAT et PLAT possèdent des débuts différents mais une terminaison identique ; lors de la construction, la structure va donc fusionner ces deux entrées au niveau de leur suffixe commun AT.
Cette approche permet de réduire drastiquement la redondance de l'information tout en garantissant une validation très rapide.

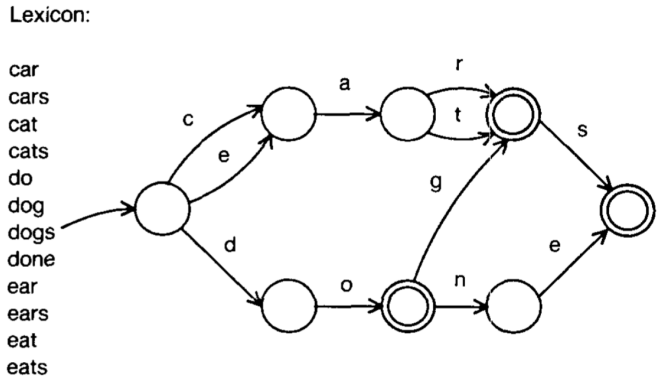


FIGURE 5 – Exemple d'une structure DAWG. [5]

2.7.1 Construction et complexité

La création d'un DAWG optimisé repose sur l'algorithme de Daciuk [10], qui impose une création des mots selon l'ordre lexicographique. Cette contrainte permet de traiter le dictionnaire de manière séquentielle : lorsqu'un nouveau mot est ajouté, l'algorithme identifie la partie de la structure qui ne sera plus modifiée et procède à la fusion des états équivalents, c'est-à-dire des nœuds possédant les mêmes transitions vers les mêmes états terminaux.
En matière de performance, la construction affiche une complexité temporelle de $O(|W| \times L)$, où $|W|$ représente le nombre total de mots et L leur longueur moyenne.

Pour la phase de jeu, la recherche d'un mot s'effectue en un temps linéaire $O(N)$ par rapport à la longueur N de la chaîne de caractères, ce qui garantit une validation quasi instantanée. Cependant, cette structure unidirectionnelle montre ses limites dès lors qu'une lettre « ancre » est déjà fixée au milieu d'un mot potentiel sur le plateau. Le DAWG ne permettant pas de remonter le chemin vers la racine pour identifier les débuts de mots possibles, le moteur de jeu est alors contraint d'explorer aveuglément tous les préfixes imaginables avant d'atteindre l'ancre, ce qui rend la génération de coups particulièrement inefficace dans ces configurations.

2.7.2 Avantages et limites pour le Scrabble

L'intérêt majeur du DAWG réside dans sa compacité exceptionnelle, permettant de compresser les centaines de milliers de mots d'un dictionnaire en un volume de données souvent inférieur à deux mégaoctets. Cependant, si cette structure est excellente pour la validation d'un mot saisi par un utilisateur, elle s'avère plus limitée pour la phase de génération automatique de coups. En raison de son parcours strictement unidirectionnel, de la racine vers les feuilles, le DAWG ne peut pas explorer efficacement les possibilités de jeu s'articulant autour d'une ancre déjà présente sur la grille.

Pour placer un mot contenant une lettre fixe en son centre ou à sa fin, l'algorithme est incapable de remonter le graphe pour identifier les préfixes compatibles. Il est alors contraint de tester à l'aveugle une multitude de combinaisons de lettres en espérant atteindre l'ancre, ce qui provoque une inefficacité notoire face à l'explosion combinatoire des tirages. Cette incapacité à traiter nativement la recherche bidirectionnelle souligne la nécessité d'une structure plus évoluée, capable de s'affranchir de la contrainte du parcours de gauche à droite : le GADDAG.

2.8 Le GADDAG — la solution pour le Scrabble

Le GADDAG, conçu par Steven Gordon en 1994 [6], a donc apporté une réponse définitive au problème de l'ancrage en permettant une recherche bidirectionnelle efficace. Contrairement au DAWG qui ne stocke qu'une version linéaire de chaque mot, le GADDAG génère n représentations différentes pour un mot de longueur n, une pour chaque position possible d'une lettre sur le plateau. La structure repose sur une transformation : pour chaque lettre choisie comme ancre, la partie située à sa gauche est inversée, puis séparée de la partie droite par un caractère spécial «>» servant de délimiteur.

À titre d'exemple, le mot « CHAT » est décomposé en quatre représentations distinctes :

1. **C > H A T** (l'ancre est C)
2. **H C > A T** (l'ancre est H : on place C à gauche, puis on complète A-T à droite)
3. **A H C > T** (l'ancre est A : on place C et H à gauche, puis T à droite)
4. **T A H C >** (l'ancre est T : on complète tout le mot vers la gauche)

Cette méthode se généralise sans perte d'efficacité pour des mots plus complexes comme « SCRABBLE », qui produira huit entrées allant de « S>CRABBLE » jusqu'à la forme totalement inversée « ELBBARCS> ». Cette multiplication des chemins garantit que, pour n'importe quelle lettre du mot déjà présente sur la grille, il existe un chemin unique dans l'automate commençant par celle-ci et permettant de créer le reste du mot.

2.8.1 Semi-minimisation

L'augmentation du nombre de représentations par mot pourrait laisser craindre une explosion de la consommation mémoire, mais le GADDAG compense ce volume par une technique dite de semi-minimisation. Gordon a démontré que si les préfixes inversés divergent souvent, les suffixes situés après le caractère pivot « > » présentent une redondance massive. La semi-minimisation consiste donc à fusionner tous les états équivalents rencontrés après le délimiteur, exactement comme dans la construction d'un DAWG.

Pour le mot « CARE », les formes « AC>RE » et « RA>CE » voient leurs branches fusionnées avec n'importe quel autre mot du dictionnaire se terminant par « RE » ou « CE ». De même, des mots partageant des suffixes verbaux, comme « MARCHER » et « JOUER », partageront une structure commune pour leurs formes pivotées au-delà du délimiteur (par exemple « RAHC>MER » et « UOJ>ER » où la terminaison « ER » est mutualisée). Cette optimisation rend la structure déterministe et permet de conserver un seul chemin par couple (mot, ancre), tout en maintenant une empreinte mémoire tout à fait acceptable pour les systèmes modernes.

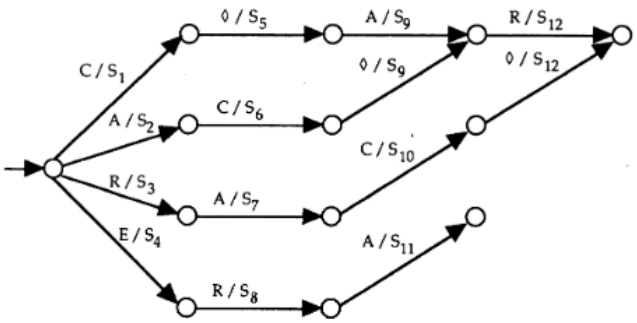


FIGURE 6 – Graphe du GADDAG semi-minimisé pour « CARE ». (Source : [6])

2.8.2 Utilisation dans le moteur de jeu

L'intégration du GADDAG au sein du moteur de jeu transforme radicalement la génération de coups en la rendant purement itérative. Pour chaque case « ancre » identifiée sur le plateau, l'algorithme entame un parcours du graphe en deux phases distinctes. Dans un premier temps, il explore les branches correspondant à la partie gauche du mot (le préfixe inversé) en remontant vers les cases vides à gauche ou en haut de l'ancre. Dès qu'il rencontre le caractère « > » dans l'automate, il bascule dans la seconde phase pour compléter le mot vers la droite ou vers le bas avec les lettres restantes du tirage.

Cette approche permet de découvrir l'intégralité des coups légaux en un seul passage dans l'automate, sans jamais avoir besoin de revenir en arrière ou de deviner une position de départ. En éliminant les tests de préfixes inutiles propres au DAWG, le GADDAG permet à l'IA d'évaluer des milliers de possibilités par seconde, garantissant une réactivité optimale même face à des dictionnaires extensifs et des contraintes de temps réelles.

2.9 Analyse du benchmark et justification du choix

TABLE 2 – Benchmark DAWG vs GADDAG — Lexique : 178 691 mots

Structure	Temps de chargement
DAWG	624 ms
GADDAG	1 443 ms

Mot	Existe	DAWG	GADDAG
APPLE	vrai	3,79 µs	8,91 µs
ZYZZYVA	vrai	4,32 µs	9,70 µs
SCRABBLE	vrai	3,96 µs	8,08 µs
JAVA	vrai	2,75 µs	8,20 µs
INEXISTANT	faux	4,19 µs	13,94 µs

Ancre	Rack	DAWG	GADDAG	Coups (D)	Coups (G)
E	RSTLNA	2,966 ms	4,752 ms	170	170
S	CRABBLE	1,184 ms	1,288 ms	112	112
Z	AEIOUST	0,467 ms	0,269 ms	16	16
X	AILNOR	0,289 ms	0,178 ms	11	11

Afin de justifier notre choix, nous avons mis en place un benchmark comparatif entre les deux structures, conduit sur notre lexique de 178 691 mots. Nos résultats révèlent une image nuancée qui justifie pleinement le choix du GADDAG comme structure centrale du moteur de jeu..

Le GADDAG accuse un temps de chargement plus élevé (1 443 ms contre 624 ms pour le DAWG), ce qui s'explique directement par sa transformation structurelle : chaque mot de longueur n génère n représentations distinctes, alourdissant mécaniquement la phase de construction. De même, sur la validation isolée d'un mot (*contains*), le DAWG reste environ deux fois plus rapide (3 à 4 µs contre 8 à 14 µs). Ces écarts sont cependant sans conséquence pratique : ces opérations sont réalisées une seule fois à l'initialisation ou de manière ponctuelle, et les valeurs absolues restent négligeables à l'échelle d'une partie.

C'est sur la génération de coups que le choix se justifie pleinement. Les résultats montrent une tendance claire : plus l'espace de jeu est contraint (peu de mots possibles), plus le GADDAG surpasse le DAWG. Pour l'ancre Z avec le rack AEIOUST (16 coups trouvés), le GADDAG est 42 % plus rapide (0,269 ms contre 0,467 ms) ; pour X avec AILNOR (11 coups), il est 38 % plus rapide (0,178 ms contre 0,289 ms). Ces cas correspondent précisément aux situations les plus fréquentes en fin de partie ou sur des lettres rares, où l'espace combinatoire est restreint mais où chaque coup compte. En fin de partie, le plateau étant déjà largement occupé, les ancres disponibles sont nombreuses

mais les extensions possibles réduites : le GADDAG, en naviguant directement depuis chaque ancre sans exploration aveugle des préfixes, évite une explosion combinatoire inutile et identifie les rares coups légaux restants bien plus efficacement que le DAWG, qui continuerait à tester des chemins stériles.

Sur les racks plus généreux (ancre E avec RSTLNA, 170 coups), le DAWG reprend l'avantage. Cela s'explique par le fait que dans ce cas, la quantité de chemins à explorer est si importante que le surcoût structurel du GADDAG se ressent. Toutefois, les deux structures trouvent exactement le même nombre de coups dans tous les cas testés, ce qui confirme la correction de l'implémentation.

	DAWG		GADDAG		Ratio
	overall	per move	overall	per move	(D/G)
CPU time					
Expanded	9:32:44	1·344s	3:38:59	0·518s	2·60
Compressed	8:11:47	1·154s	3:26:51	0·489s	2·36
Page faults					
Expanded	6063		32,305		0·19
Compressed	1011		3120		0·32
Arcs Traversed	668,214,539	26,134	265,070,715	10,451	2·50
Per sec (compressed)	22,646		21,372		
Anchors used	3,222,746	126·04	1,946,163	76·73	1·64
Number of moves	25,569		25,363		
Average score	389·58		388·75		

FIGURE 7 – Benchmark original de S.Gordon en 1994. (Source : [6])

À titre de comparaison, Gordon [6] avait mesuré sur 1000 parties aléatoires que le GADDAG réduisait le temps CPU d'un facteur 2,5 environ par rapport au DAWG, avec 1 946 163 ancres traitées contre 3 222 746 pour le DAWG (ratio de 1,64). Bien que l'écart mesuré dans notre benchmark soit moins prononcé, ces résultats confirment l'avantage structurel du GADDAG sur la génération de coups.

Conclusion du choix. Le DAWG est une structure excellente pour la validation de mots, mais son parcours strictement unidirectionnel le contraint à explorer aveuglément des préfixes inutiles dès qu'une ancre est présente sur la grille. Le GADDAG, en encodant toutes les positions d'ancrage possibles, élimine ce surcoût algorithmique et offre des performances supérieures précisément là où cela importe le plus : les situations tendues, comme les fins de partie au Scrabble. Le léger surcoût en mémoire et en temps de chargement est un compromis largement acceptable au regard du gain en efficacité durant le jeu.

3 Architecture du projet

Le projet est organise en couches fonctionnelles strictement separees selon le patron MVC. Chaque package porte une responsabilité unique, avec des dépendances majoritairement descendantes.

3.1 Découpage en modules

Point d'entrée La classe `App` initialise le contexte d'exécution : détection de la langue via `I18n`, chargement de la configuration `.scrabblerc` via `ConfigLoader`, puis délégation au mode d'affichage selon les options `commons-cli`. `App` expose des `@FunctionalInterface` (`CliLauncherHandler`, `GuiLauncherHandler`, `ExitHandler`) pour découpler le point d'entrée des implémentations concrètes et faciliter les tests.

Couche contrôleur Le package `controller` contient `GameController` (logique applicative principale) et `GameControllerAux` (orchestration CLI extraite pour limiter la taille de la classe principale). `GameController` maintient les références au modèle (`Game`, `Gaddag`), à la vue (`UserInterface`) et aux paramètres de lancement (mode blitz, super-scrabble, IA). Il dépend également de sous-contrôleurs spécialisés : `builders/` pour la construction interactive des coups, `config/` pour les snapshots de configuration, `dictionary/` pour le chargement du dictionnaire, et `network/` pour les commandes réseau CLI.

Couche métier — core Le package `model.core` regroupe les objets de domaine et les règles de jeu. `Game` est le moteur central : il gère le plateau (`Board`), le sac (`Bag`), la liste des joueurs, le tour courant, le mode blitz (minu-

terie par joueur) et l'historique undo/redo. `MoveHandler` est le seul composant autorisé à modifier l'état du jeu en réponse à un `Move`. `Scoring` calcule les points (multiplicateurs de cases, bonus Bingo à 50 points si 7 tuiles posées). `MoveGenerator` génère la liste des coups jouables pour l'IA. `UndoRedo` gère deux piles (`Stack<Move>`) pour l'annulation et le rejeu.

Couche métier — dictionnaire Le package `model.dictionary` expose une interface `Dictionary` (méthodes `add`, `contains`, `findWordsWithRackAndHook`) implémentée par trois structures : `Trie` (recherche préfixe générique), `Dawg` (automate acyclique déterministe, optimisé pour la validation rapide) et `Gaddag` (structure bidirectionnelle indispensable pour la génération de coups). Le contrôleur et l'IA travaillent exclusivement avec `Gaddag`, qui est chargé une seule fois depuis le fichier dictionnaire selon la langue active.

Couche métier — IA Le package `model.ai` regroupe `AiPlayer` (sous-classe concrète de `Player`, surcharge `isAutonomous()` et `playTurn()`), `MinimaxSolver` (Minimax et Expectiminimax avec profondeur et limite de temps configurables, 200 tirages de simulation par nœud) et `MlAgent` (agent TensorFlow Java, chargement gracieux : si le modèle est absent, l'IA se replie sur le Minimax sans crash).

Couche métier — joueurs La classe abstraite `Player` (dans `model.interfaces`) unifie `HumanPlayer` et `AiPlayer`. Elle encapsule le nom, le score, le `Rack` et la minuterie blitz. Les méthodes principales sont `enableBlitzClock()`, `startTurnTimer()`, `pauseTurnTimer()` et `isOutOfTime()`. `HumanPlayer` n'ajoute aucune logique : il hérite simplement de `Player` sans surcharger `isAutonomous()` (retourne `false` par défaut). Un joueur réseau est représenté côté client par un `Player` standard dont les coups sont pilotés par `GameClient`.

Couche vue Le package `view` expose l'interface `UserInterface`. Elle définit quatre méthodes : `refresh()`, `displayMessage()`, `displayError()` et `displaySuccess()`. Deux implémentations concrètes existent : `CliView` (rendu ANSI via `BoardRenderer`, `RackRenderer`, `PlayerRenderer`, `MessageRenderer`) et `JavaFxView` (interface graphique JavaFX avec `BoardPanel`, `RackPanel`, `ScorePanel`, `ControlPanel`, `MessagePanel`). Les launchers `CliLauncher` et `GuiLauncher` dans `view/optionlancement/` sont le point de transition entre `App` et les vues concrètes.

Persistance Le package `model.savefiles` contient `ConfigLoader` (lecture/écriture du fichier `.scrabblerc` au format INI, section `[defaults]`, commentaires # ignorés, création automatique si absent), `SaveManager` (sérialisation de l'état de jeu en trois blocs ASCII : `[settings]`, `[game]`, `[history]`) et `GameLoader` (désérialisation et reconstruction d'un `Game` depuis un fichier de sauvegarde).

Réseau Le package `model.network` est organisé en façade (`NetworkManager`), client (classe `GameClient`) et serveur (classes `GameServer`, `ClientHandler`, `OnlineGame`). `NetworkManager` maintient une liste de `NetworkObserver` (CLI et GUI s'y enregistrent). Il orchestre `GameServer` (TCP 12345, threads par client, liste synchronisée) et `DiscoveryService` (UDP 12346, broadcast de découverte automatique). Les échanges sont décodés via `PacketParser`.

Internationalisation La classe `I18n` (dans le package `i18n`) est un singleton synchronisé backed par `ResourceBundle`. Elle expose `setLanguage(lang)`, `getLanguage()` et `translate(key,args...)`. Deux langues sont supportées : `"en"` (défaut) et `"fr"`. La langue conditionne le chargement du dictionnaire et la distribution des lettres dans `Bag`.

3.2 Diagramme UML global

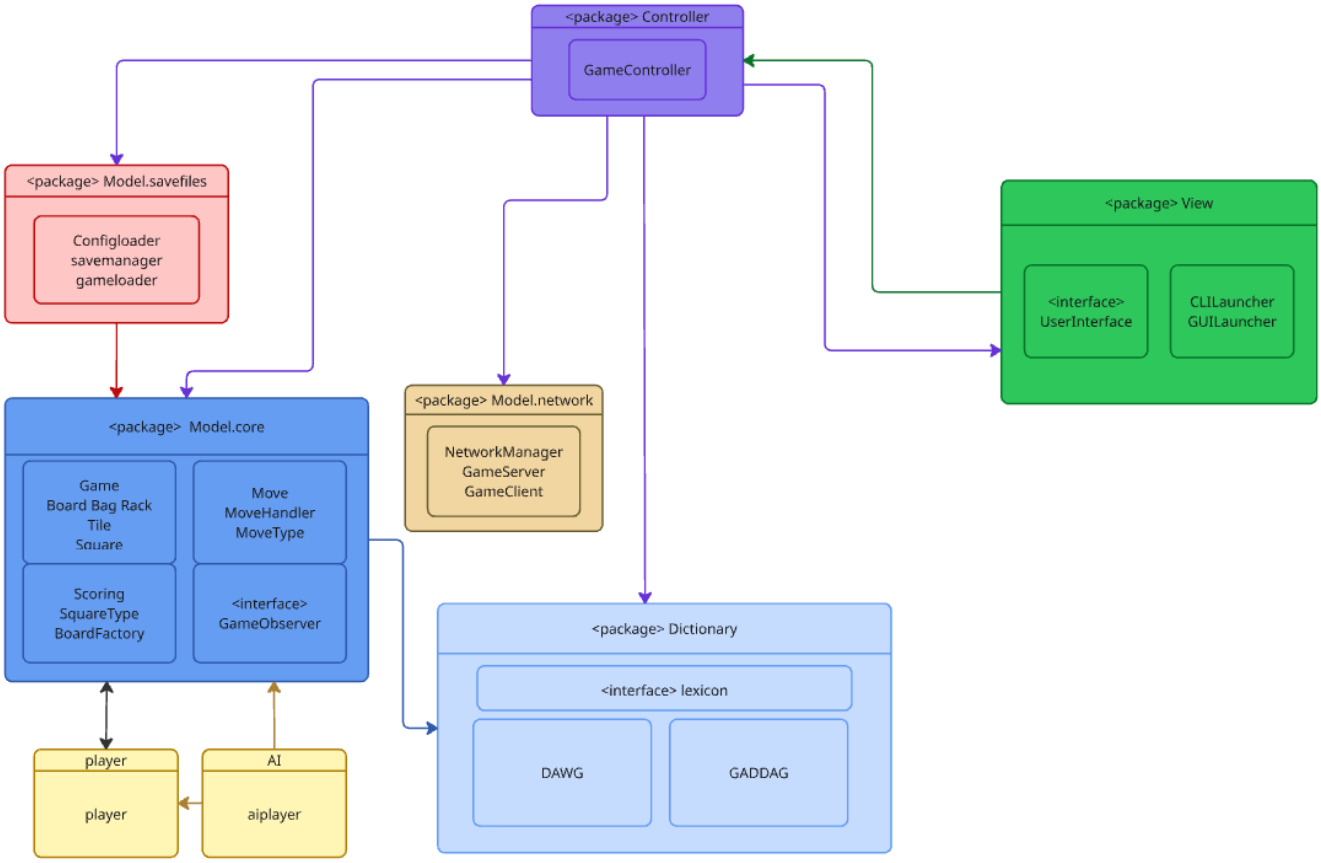


FIGURE 8 – Diagramme de l'architecture du projet.

3.3 Principales interfaces et points d'extension

Trois interfaces structurent l'évolutivité du projet.

`UserInterface` — contrat commun des vues CLI et JavaFX

```java
// view/UserInterface.java
public interface UserInterface {
    void refresh();                        // Rafraichit l'affichage complet
    void displayMessage(String message); // Message informatif
    void displayError(String error);     // Message d'erreur
    void displaySuccess(String message); // Message de succès
}
```

`CliView` et `JavaFxView` implémentent cette interface. `GameController` ne connaît que `UserInterface` — il ignore quelle implémentation est active. Ajouter une vue web revient à implémenter cette seule interface.

`Dictionary` — contrat interchangeable entre Trie, Dawg et Gaddag

```java
// model/dictionary/Dictionary.java
public interface Dictionary {
    void add(String word);
    default void addAll(String[] words) { for (String w : words) add(w); }
    boolean contains(String word);
    Set<String> findWordsWithRackAndHook(Character[] rack, char hook);
}
```

En pratique, le contrôleur et l'IA n'instancient que `Gaddag` (qui implémente `Dictionary`). Substituer une autre structure lexicale ne nécessite qu'une nouvelle implémentation de cette interface.

`Player` — abstraction commune des joueurs humains et IA

```java
// model/interfaces/Player.java (extraits)
public abstract class Player {
    protected String name;
    protected int score;
    protected Rack rack;

    public boolean isAutonomous() { return false; }       // surcharge dans AiPlayer
    public void playTurn(Game game, Gaddag gaddag) {      // surcharge dans AiPlayer
        throw new UnsupportedOperationException();
    }
    public void enableBlitzClock(Duration initialTime) { ... }
    public void startTurnTimer() { ... }
    public void pauseTurnTimer() { ... }
    public boolean isOutOfTime() { ... }
}
```

`HumanPlayer` hérite directement sans surcharge. `AiPlayer` surcharge `isAutonomous()` (retourne `true`) et `playTurn()` (délègue à `MinimaxSolver`). Le moteur de jeu appelle `player.isAutonomous()` à chaque tour pour décider s'il doit attendre la saisie utilisateur ou déclencher le jeu automatique.

`NetworkObserver` — pattern Observer pour les événements réseau

```java
// model/network/NetworkObserver.java (extraits)
public interface NetworkObserver {
    void localModelUpdate();
    void gameEndedUpdate(List<Map<String, String>> finalScore);
    void serverStatusUpdate(Map<String, String> info);
    void playersUpdate(List<Map<String, String>> players);
    void invitationReceivedUpdate(String from);
    void clientDisconnectedUpdate(String reason);
    void moveRefusedUpdate(String reason);
    // ... 15 méthodes au total, une par type d'événement réseau
}
```

`CliNetworkBridge` et `NetworkGameBridge` (côté GUI) implémentent cette interface et s'enregistrent auprès de `NetworkManager`. Ajouter un nouveau type de client réseau revient à implémenter `NetworkObserver`.

3.4 Flux d'exécution principal

Le déroulement d'un tour illustre comment les couches collaborent :

```
App
\-\-> CliLauncher.launch() / GuiLauncher.launch()
  \-\-> GameController(game, view)
    |-\-> I18n.setLanguage(lang)
    |-\-> ConfigLoader.loadConfig()         -- lecture de .scrabblerc
    |-\-> Gaddag.load(dictionaryFile)       -- chargement du dictionnaire
    |-\-> Game.addPlayer(HumanPlayer | AiPlayer)
    |-\-> Game.startGame()                  -- distribution des Rack depuis Bag
    \-\-> [boucle de jeu]
      |-\-> si player.isAutonomous()
      |     \-\-> AiPlayer.playTurn(game, gaddag)
      |           \-\-> MinimaxSolver.findBestMove(game, gaddag)
      |                 \-\-> MoveGenerator.getPlayableWordsList(
      |                       board, rack, gaddag)
      |-\-> sinon : UserInterface.refresh() + saisie CLI/GUI
      |-\-> MoveHandler.applyMove(move)
      |     |-\-> Gaddag.contains(word)       -- validation lexicale
      |     |-\-> Scoring.calculateWordScore()
      |     |                          -- calcul des points
```

```
      |     |-\-> Board.place(tile, position)  -- mise a jour du plateau
      |     |-\-> Bag.draw()                   -- recharge du Rack
      |     \-\-> UndoRedo.addMove(move)       -- empilement pour undo
      |-\-> UserInterface.refresh()             -- rafraichissement de la vue
      \-\-> Game.nextTurn()                     -- rotation des joueurs
```

Les dépendances sont strictement descendantes : la vue ne connaît que `UserInterface`, le contrôleur ne connaît que les interfaces `Dictionary`, `UserInterface` et `Player`. `Game` et `MoveHandler` ignorent totalement l'existence de la vue et du réseau.

3.5 Gestion des coups et pattern Command

Chaque action de jeu est encapsulee dans un objet `Move` construit par factory methods statiques :

```java
// model/core/Move.java (extraits)
public class Move {
    private final Player player;
    private final MoveType type;          // PLAY, EXCHANGE, PASS
    private final List<Tile> tiles;
    private final Point startPosition;    // null pour PASS et EXCHANGE
    private final Direction direction;    // null pour PASS et EXCHANGE

    // Données pour undo/redo (mutables, renseignees par MoveHandler)
    private int scoreGained;
    private List<Tile> drawnTiles;
    private List<Point> placedPositions;
    private List<Tile> placedTiles;

    public static Move createPass(Player player) { ... }
    public static Move createExchange(Player player, List<Tile> tiles) { ... }
    public static Move createPlay(Player player, List<Tile> word,
                                  Point startPosition, Direction direction) { ... }
}
```

```java
public enum MoveType { PLAY, EXCHANGE, PASS }
```

`MoveHandler` est le seul executeur : il valide le coup, met à jour `Board`, `Bag` et les scores, puis renseigne les champs mutables de `Move` (positions reellement posées, tuiles piochees, score gagne) pour permettre l'annulation. `UndoRedo` maintient deux `Stack<Move>` : `history` et `redoStack`. `addMove()` vide toujours `redoStack` (nouvelle branche d'historique).

```java
// model/core/UndoRedo.java
public class UndoRedo {
    private final Stack<Move> history;
    private final Stack<Move> redoStack;

    public void addMove(Move move) { history.push(move); redoStack.clear(); }
    public Move undo() { Move m = history.pop(); redoStack.push(m); return m; }
    public Move redo() { Move m = redoStack.pop(); history.push(m); return m; }
}
```

3.6 Génération du plateau et pattern Factory

`StandardBoardFactory` applique le pattern Factory pour créer le plateau sans que le moteur ne connaisse les coordonnées des bonus :

```java
// model/factories/StandardBoardFactory.java (extraits)
public class StandardBoardFactory {
    public static Board createBoard() { return createBoard(Board.SIZE); } // 15x15

    public static Board createBoard(int size) {
        Square[][] squares = new Square[size][size];
```

```
        // 1. Remplissage NORMAL
        for (int x = 0; x < size; x++)
            for (int y = 0; y < size; y++)
                squares[x][y] = new Square(new Point(x, y), SquareType.NORMAL);
        // 2. Application des bonus (coordonnées mises à l'echelle pour 21x21)
        applyBonuses(squares, size);
        return new Board(squares, size);
    }

    private static int scaleCoordinate(int base, int targetSize) {
        return (int) Math.round(base * (targetSize - 1.0) / (Board.SIZE - 1.0));
    }
}
```

`SquareType` est une enumeration a cinq valeurs, chacune portant ses multiplicateurs :

```
public enum SquareType {
    NORMAL(1, 1), DOUBLE_LETTER(2, 1), TRIPLE_LETTER(3, 1),
    DOUBLE_WORD(1, 2), TRIPLE_WORD(1, 3);

    private final int letterMultiplier;
    private final int wordMultiplier;
}
```

Les coordonnées de bonus du plateau 21x21 sont calculees par interpolation lineaire depuis les coordonnées du plateau 15x15 (`scaleCoordinate`), ce qui permet de supporter les deux variantes sans dupliquer les données de configuration. `Game` choisit la taille du plateau selon le `GameMode` recu à la construction : `new Board(21)` pour `GameMode.SUPER`, `new Board()` (taille par défaut 15) pour `GameMode.STANDARD`.

4 Explication approfondie

4.1 Spécification Étendue - F21 : Sauvegarde et restauration d'une partie

Description du besoin :

Le programme doit pouvoir sérialiser une partie en cours dans un fichier texte ASCII et reconstruire une partie complète à partir d'un fichier de sauvegarde. Le parseur doit être robuste aux erreurs de format et les signaler avec précision (type d'erreur et numéro de ligne). L'implémentation repose sur les classes `SaveManager.java` et `GameLoader.java`.
Le format retenu est volontairement simple et lisible : trois blocs principaux `[settings]`, `[game]` et `[history]`, complétés par des paires clé/valeur. La restauration suit un vrai enchaînement de parsing : une prélecture détermine d'abord le mode de jeu à reconstruire, puis le fichier est parcouru ligne par ligne avec séparation des commentaires, identification de la section courante et validation des champs. Toute erreur est remontée avec un numéro de ligne afin de faciliter le diagnostic.

Métadonnées :
- **Développeur responsable :** Thomas Codognan
- **Date de livraison prévue :** Semaine 7
- **Dépendances inversées :** F22 (Commentaires), F23 (Settings), F24 (Plateau), F25 (Historique)

4.1.1 Type de besoin

Fonctionnel. Ce besoin permet la persistance d'une partie entre deux sessions de jeu.

4.1.2 Prérequis

- **F2 : Configuration** : Paramètres par défaut via `.scrabblerc`, gérés par `ConfigLoader.java`.
- **F10 : Règles du jeu** : État complet du jeu (plateau, scores, joueurs, historique, tour courant).
- **F15 : Shell interactif** : Commandes `save FILE` et `load FILE`, pilotées par `GameControllerAux.java`.

4.1.3 Découpage en sous-besoins et détails d'implémentation

Sous-besoin 1 : Sérialisation complète d'une partie

Fonction associée :

— Sauvegarde : `void saveGame(Game game, String filePath) throws IOException`

`SaveManager` est l'unique point de sérialisation. Lorsque l'utilisateur exécute `save FILE`, le contrôleur auxiliaire délègue immédiatement à cette méthode. Elle ouvre (ou crée) le fichier cible via un `PrintWriter`, puis écrit successivement les trois sections obligatoires dans l'ordre : `[settings]`, `[game]`, `[history]`. Chaque section est produite par une méthode interne dédiée, ce qui garantit qu'une erreur d'écriture dans l'une d'elles remonte une `IOException` propre sans laisser de fichier partiellement écrit sur le disque.

Cas	Entrée	Résultat attendu
`testSaveCompleteGame()`	État complexe : plateau avec mots, 2 joueurs, scores, racks, coups pass/play.	Fichier créé avec les 3 sections, vérifié via `SaveManagerTest.java`.
`testSaveEmptyGame()`	Partie vide, aucun coup joué.	Plateau sérialisé en lignes de tirets, section `[history]` présente mais vide.
`testSaveCoordinateFormat()`	Conversion de coordonnées `a1v` / `h8h`.	Format des coordonnées correct dans `[history]`.

TABLE 3 – Tests unitaires de la sérialisation complète

Sous-besoin 2 : Sérialisation des sections `[settings]` et `[game]`

Fonctions internes :

— `writeSettings(PrintWriter writer, Game game)`

— `saveBoard(PrintWriter writer, Board board)`

— `serializeRack(Player p)`

La section `[settings]` regroupe les paramètres globaux de la partie et les attributs IA joueur par joueur : `blitz`, `super-scrabble`, `players-count`, `debug/verbose`, `player-N-type` (`human` ou `ai`), `player-N-ai-mode` et `player-N-name`. La section `[game]` encode ensuite l'état vivant de la partie : la première ligne indique l'index du joueur courant (1-based), puis le plateau est écrit ligne par ligne (lettre majuscule ou tiret – pour case vide), suivi des clés `rack-N` et `score-N` pour chaque joueur, et de la clé `language`, non stockée dans `[settings]` pour des raisons de chargement anticipé.

Cas	Entrée	Résultat attendu
`testSaveBoardWord()`	Plateau avec le mot `COUNT` en `g5h`.	La ligne `g` contient `C O U N T` aux bonnes colonnes.
`testSaveEmptyBoard()`	Plateau entièrement vide.	Toutes les cases représentées par `-`.
`testSaveScores()`	Joueur 1 score 15 / joueur 2 score 10.	Présence de `score-1: 15` et `score-2: 10`.
`testSaveAiSettings()`	Joueur 2 de type IA, mode Expectiminimax.	Présence de `player-2-type=ai` et `player-2-ai-mode=Expectiminimax`.

TABLE 4 – Tests unitaires des sections `[settings]` et `[game]`

Sous-besoin 3 : Sérialisation de la section `[history]`

Fonctions internes :

— `saveHistory(PrintWriter writer, Game game, List<Move> history)`
— `readFullWordFromBoard(Board board, Move move)`
— `convertPointToCoord(Point p)`

L'historique est écrit dans l'ordre chronologique. Chaque coup est converti en une ligne selon son type : `N pass` pour un passage, `N exchange ABC` pour un échange de lettres, et `N g5h WORD` ou `N e6v WORD` pour un coup normal horizontal ou vertical. La méthode `readFullWordFromBoard` est nécessaire pour les coups qui s'appuient sur des lettres déjà présentes sur le plateau : elle reconstitue le mot complet depuis le plateau en distinguant par la casse les lettres posées lors de ce coup précis des lettres déjà en place.

Cas	Entrée	Résultat attendu
`testSaveHistoryPlayMoves()`	Joueur 1 joue `COUNT` en `g5h`, joueur 2 joue `PHOTO` en `e6v`.	Présence de `1 g5h COUNT` et `2 e6v PHoTO` en tête d'historique.
`testSaveHistoryPass()`	Coup de type `pass`.	Format `N pass` écrit correctement.
`testSaveHistoryExchange()`	Coup de type `exchange`.	Format `N exchange LETTRES` écrit correctement.
`testSaveEmptyHistory()`	Aucun coup joué.	Section `[history]` présente, sans ligne de coup.

TABLE 5 – Tests unitaires de la section `[history]`

Sous-besoin 4 : Chargement et reconstruction d'une partie

Fonction associée :

— Chargement : `Game loadGame(String filePath) throws Exception`

`GameLoader` effectue une prélecture du fichier pour détecter le flag `super-scrabble` et instancier le bon plateau (15×15 ou 21×21) avant tout parsing. Le fichier est ensuite parcouru ligne par ligne via un `BufferedReader` : les commentaires sont supprimés, la section courante est identifiée par les balises `[...]`, puis chaque ligne est dispatchée vers `parseSettings`, `parseGameState` ou `parseHistory` selon la section active. Toute anomalie sémantique (valeur non numérique pour un score, section absente, coordonnée invalide) est remontée sous la forme `Format error at line X: <détail>`. L'état courant du jeu n'est jamais écrasé en cas d'échec.

```
while ((line = reader.readLine()) != null) {
    lineCount++;
    line = processLineComments(line).trim();
    if (line.isEmpty() || isInBlockComment) continue;

    if (line.startsWith("[")) {
        section = line.toLowerCase();
        continue;
    }
    switch (section) {
        case "[settings]": parseSettings(game, line);              break;
        case "[game]":     boardRow = parseGameState(game, line, boardRow); break;
        case "[history]":  parseHistory(game, line);               break;
        default: throw new IllegalStateException(
            "Data found before any valid section header.");
    }
}
```

Cas	Entrée	Résultat attendu
`testLoadValidGame()`	Fichier valide et complet.	`Game` non nul avec plateau, scores et historique cohérents.
`testLoadSuperScrabble Detection()`	Plateau 21×21 (super-scrabble).	Détection correcte et création du bon plateau par prélecture.
`testLoadAiPlayer Settings()`	`player-1-ai-mode=Expectimiste`	Restauration fidèle d'un joueur IA avec son algorithme.
`testLoadInvalidScore()`	`score-1: abc`.	Exception `Format error at line X: ....`
`testLoadMissing GameSection()`	Section `[game]` absente.	Exception avec message explicite.
`testLoadHistoryWith Exchange()`	Coup `exchange` dans l'historique.	Coup restauré correctement, taille historique = 1.

TABLE 6 – Tests unitaires du chargement et de la reconstruction

Sous-besoin 5 : Parsing et suppression des commentaires

Fonction interne :

— `processLineComments(String line)`

Avant tout parsing sémantique, chaque ligne est nettoyée par `processLineComments`. Cette méthode supprime tout ce qui suit un `#` sur une ligne (commentaire en ligne) et gère les blocs multi-lignes `{ ... }` via un état booléen `isInBlockComment`. Le compteur `lineCount` est incrémenté avant ce traitement, ce qui garantit que les messages d'erreur référencent toujours le numéro de ligne physique du fichier source, et non un numéro décalé par les lignes de commentaires ignorées.

Cas	Entrée	Résultat attendu
`testInlineComment()`	`debug=true # activer le debug`.	Retourne `debug=true` après suppression du suffixe commenté.
`testBlockComment()`	Bloc `{ commentaire\nsur 2 lignes }` puis `language=fr`.	Bloc supprimé, `language=fr` conservé et parsé.
`testNoComment()`	Fichier sans commentaire.	Contenu strictement identique en sortie.

TABLE 7 – Tests unitaires du parsing des commentaires

4.1.4 Intégration dans l'architecture du projet

Module	Composant / Classe	Responsabilité
Persistance	`SaveManager.java`	Sérialise l'état complet du jeu en ASCII structuré en 3 sections.
Persistance	`GameLoader.java`	Parse le fichier, reconstruit le `Game` et restaure l'historique complet.
CLI	`GameControllerAux.java`	Reçoit `save FILE` / `load FILE` et délègue aux deux classes ci-dessus.
GUI	`ScrabbleGui.java` `GameController.java`	Menu *Save* → `SaveManager`. Menu *Load* → `GameLoader`, puis rebind du contrôleur et refresh de l'UI.
Configuration	`ConfigLoader.java`	Lit `.scrabblerc` au démarrage pour les valeurs par défaut avant le chargement. Note : le chemin par défaut de sauvegarde n'est pas encore pris en charge si `FILE` est absent.

TABLE 8 – Architecture de la fonctionnalité Sauvegarde/Restauration

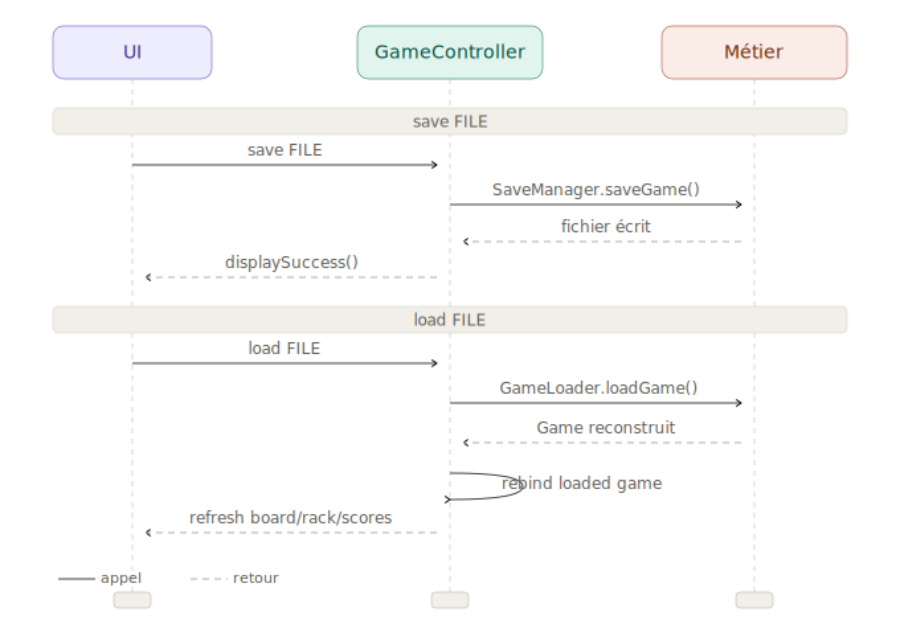


FIGURE 9 – Diagramme de séquence du scénario de sauvegarde et restauration d'une partie.

4.1.5 Scénario utilisateur

1. L'utilisateur joue plusieurs coups en CLI ou via l'interface graphique.
2. En CLI, il tape : `save ma_partie.scrabble`
3. `SaveManager` écrit `[settings]`, `[game]` puis `[history]` dans le fichier.
4. Le shell affiche un message de succès.
5. L'utilisateur relance le programme et tape : `load ma_partie.scrabble`
6. `GameLoader` reconstruit le `Game` : joueurs, plateau, tour courant et historique.
7. Si une ligne est invalide, le programme affiche `Format error at line X: ...` et n'écrase pas l'état courant.

Stratégie de validation

Validation du scénario : ce besoin est validé par des tests unitaires de round-trip. `SaveManagerTest.java` couvre la sérialisation complète d'une partie jouée et d'une partie vide, tandis que `GameLoaderTest.java` vérifie la détection du mode super-scrabble, la restauration d'un joueur IA et la reconstruction d'un historique contenant un `exchange`. Ces tests simulent le scénario utilisateur sans passer par la GUI, validant directement F21 au niveau du modèle.

```
@Test
void testLoadHistoryWithExchange() throws Exception {
    String content = "[settings]\n[game]\n1\n"
        + "---------------\n".repeat(15)
        + "[history]\n1 exchange AZERTY";
    Game game = loader.loadGame(path.toString());
    assertEquals(1, game.getUndoRedo().getHistory().size());
}
```

4.2 Spécification Étendue - F39 : Serveur Multi-clients

Description du besoin :

Le programme doit implémenter un serveur multi-clients capables de gérer plusieurs parties de Scrabble simultanément en réseau. Ce serveur devra accepter et maintenir simultanément plusieurs connexions clientes, **sans qu'aucune de ces connexions ne vienne bloquer le traitement des autres**.
Suivant un **modèle autoritaire** (Server-Authoritative), le serveur centralise la logique métier et l'état du jeu, et garantit la synchronisation en temps réel entre tous les clients.

Métadonnées :
- **Développeur responsable :** Léonard Bettcher
- **Date de livraison prévue :** Semaine 9
- **Dépendances inversées :** F40 (Invitations)

4.2.1 Type de besoin

Fonctionnel, ce besoin permet la réalisation de parties multi-joueurs en réseau.

4.2.2 Prérequis

- **F38 : Connexion TCP et protocole d'échange** : Établissement d'une liaison socket entre serveur et client. Le protocole réseau s'appuie sur une syntaxe textuelle `CLE=VALEUR` délimitée par des points-virgules (`PacketParser`).
- **F10 : Moteur de jeu** : Le fonctionnement du serveur en mode autoritaire nécessite l'accès à la logique métier définie dans `Game.java`, utilisée comme modèle de référence pour vérifier la validité des coups (grille, dictionnaire, calcul des points).

4.2.3 Découpage en sous-besoins et détails d'implémentation

Sous-besoin 1 : Architecture Multi-thread et gestion de la concurrence

Fonctions internes :

- Délégation au thread : `new Thread(handler).start()`
- Sécurisation globale : `synchronized (stateLock)`

Le serveur devra gérer des requêtes simultanées de manière asynchrone, en maintenant **l'intégrité des données partagées**.

Quand une connexion entrante est acceptée par le `GameServer`, la gestion de ce socket est déléguée à une instance de `ClientHandler`, s'exécutant dans son propre thread. Cette approche ***Thread-per-client*** garantit la non-bloquance des requêtes.

La liste des clients connectés est une **ressource hautement partagée**. Elle est créée via `Collections.synchronizedList(...)` pour garantir que les opérations d'ajout et de retrait soient **atomiques**. Cependant, cette classe ne protège pas contre les requêtes nécessitant un parcours complet de la liste. Notre code utilise donc un verrouillage manuel via un bloc `synchronized (clients)` pour prévenir toute **`ConcurrentModificationException`**.

Enfin, `OnlineGame` gère la concurrence au sein d'une partie, mais le serveur à aussi besoin d'une protection globale. On utilise un mutex privé, `stateLock`. Ce verrou est obligatoire pour le système d'invitations et protège les opérations de type ***Check-Then-Act*** (vérifier un statut, puis le modifier). Grâce à lui, il est impossible qu'une ***race condition*** soit levée (ex : Joueur A accepte une invitation à la milliseconde où Joueur B l'annule).

Cas	Entrée	Résultat attendu
`testMultiplayerInvitation WithMixedResponses()`	Connexion de 4 clients. L'hôte invite 3 joueurs (IDs 2, 3, 4). Deux acceptent, le 3ème refuse.	Le verrou d'état gère les accès concurrents. La partie démarre avec l'hôte et les 2 joueurs ayant accepté.
`testHostDisconnection DuringInvitation()`	L'hôte ferme sa connexion réseau alors qu'une invitation est en attente.	Le serveur détecte la déconnexion, annule l'invitation en cours et renvoie une erreur aux autres joueurs.

TABLE 9 – Tests unitaires de l'architecture multi-thread

Sous-besoin 2 : Émission (Client) et Parsing Syntaxique (Serveur)

Fonctions associées :

- Initialisation du parser : `public PacketParser(String rawMessage)`
- Émission client : `public void sendPlayMove(int x, int y, String direction, String tile)`

Via la commande `new`, le serveur instancie l'objet `OnlineGame`. C'est la **référence unique d'une partie réseau**, qui encapsule le modèle de jeu autoritaire et regroupe les flux de communication. Pour illustrer le cycle de vie d'une requête, prenons la commande `MOVE`. L'appel est fait par `NetworkManager`, une **façade simplifiant les interactions** entre l'interface et le réseau. Lorsqu'un joueur valide son coup, le `GameClient` va **sérialiser cette action** en une chaîne respectant le protocole (`MOVE:TYPE=PLAY;X=7;Y=7;DIR=H;TILES=LA`) et l'envoyer sur le socket.

À la réception, le thread du `ClientHandler` délègue **l'extraction des informations** à la classe `PacketParser`. Ce parseur effectue un découpage en quatre étapes :

1. **En-tête :** Isoler la commande via les deux-points `:` (extraction de `MOVE`).
2. **Entités :** Découper le *payload* selon le caractère pipe `|`.
3. **Attributs :** Isoler les arguments en s'appuyant sur le point-virgule `;`.
4. **Affectation :** Séparer Clé/Valeur avec le signe égal =.

Cas	Entrée	Résultat attendu
`testProcessPlayMove()`	Instanciation manuelle du `PacketParser` avec la chaîne : `MOVE:TYPE=PLAY;X=7...`	Le parseur découpe la trame sans erreur, permettant d'extraire la direction, les coordonnées et les tuiles.

TABLE 10 – Tests unitaires du parsing syntaxique

Sous-besoin 3 : Validation Sémantique et Modèle Autoritaire

Fonctions associées :

— Réception et aiguillage : `public synchronized void processMove(...)`

Le serveur doit traduire la requête réseau en une action de jeu concrète. C'est ici que s'exprime le paradigme du modèle autoritaire (Server-Authoritative), essentiel pour maintenir l'intégrité de l'état du jeu et prévenir la triche **[11]** : **le client propose, le serveur dispose**.
Avant d'instancier l'action, le serveur effectue un **parsing sémantique sécurisé**. La conversion des valeurs "X" et "Y" en `Integer` est protégée par des blocs `try-catch`. Si un client malveillant envoie des coordonnées erronées, l'exception est interceptée, **évitant le crash du thread**, et une erreur protocolaire lui est renvoyée.

Si les données sont saines, `OnlineGame` va **faire le pont entre le réseau et le moteur du jeu**. Les entiers deviennent un `Point(7, 7)`, la chaîne devient une `Direction`, etc. Cela permet d'instancier un objet `Move`.

Cet objet `Move` est testé par le moteur de jeu central (`Game`), qui **vérifie sa validité linguistique et légale**. Si le coup est illégal, l'opération est annulée. S'il est valide, le serveur met à jour **son** plateau de référence et recalcule les scores.

Cas	Entrée	Résultat attendu
`testProcessMove WrongTurn()`	Le Joueur 2 envoie une trame `MOVE` alors que c'est le tour du Joueur 1.	Le modèle autoritaire bloque l'action et renvoie exclusivement au Joueur 2 : `err_not_your_turn`.
`testProcessPlayMove()`	Le Joueur 1 place le mot valide "LA" aux coordonnées de départ (7, 7).	Le coup est exécuté sur le modèle du serveur et le tour passe au Joueur 2.

TABLE 11 – Tests unitaires du modèle autoritaire

Sous-besoin 4 : Synchronisation par Diffusion et Asymétrie

Fonctions associées :

— Diffusion générale : `public void broadcast(String message)`
— Diffusion ciblée : `public void sendRack(ClientHandler handler, Player player)`

Une fois le coup validé, les clients sont désynchronisés. `OnlineGame` va itérer sur sa liste locale de `ClientHandler` pour envoyer les paquets de mise à jour. À la réception, les clients actualisent leur modèle miroir local, ce qui déclenche le **rafraîchissement de leur interface graphique** via le `NetworkObserver`.

Asymétrie des données (Information partielle) :
Si le serveur diffusait l'intégralité du modèle à tous les joueurs, un utilisateur malveillant utilisant un *packet sniffer* pourrait intercepter les lettres de ses adversaires. Pour résoudre ce problème, le serveur implémente une **diffusion asymétrique** :

— **Données publiques :** Le plateau mis à jour (`OPPONENT_MOVE`) et l'évolution des scores sont envoyés à l'ensemble des participants.
— **Données privées :** Seul le joueur qui vient de jouer reçoit un paquet `SET_RACK` contenant ses nouvelles lettres.

Grâce à cette séparation, le réseau ne transporte **que les données strictement nécessaires**. Le rack des adversaires est sécurisé au niveau même de l'architecture réseau.

Cas	Entrée	Résultat attendu
`testProcessPlayMove()`	Le processus de diffusion asymétrique s'enclenche après un mot valide.	Le Joueur 2 reçoit les données publiques (`OPPONENT_MOVE`). Le Joueur 1 reçoit ses données privées (`SET_RACK`).
`testRealMultiplayer Interaction()`	Le client reçoit la mise à jour de la grille du serveur.	Le modèle miroir est mis à jour et déclenche `localModelUpdate()` pour rafraîchir l'UI.

TABLE 12 – Tests de synchronisation réseau

4.2.4 Intégration dans l'architecture du projet

Module	Composant / Classe	Responsabilité
Point d'entrée (Façade)	`NetworkManager.java`	Gère le cycle de vie des instances client/serveur et simplifie les interactions de l'UI.
Réseau (Serveur)	`GameServer.java`	Écoute TCP. Gère la boucle principale (`ServerSocket`) et déploie les threads d'écoute.
Réseau (Liaison)	`ClientHandler.java`	Isolation. Classe dédiée à un client unique. Assure la lecture non-bloquante et gère les timeouts.
Logique Réseau (Modèle)	`OnlineGame.java`	Modèle autoritaire. Encapsule les règles du Scrabble, valide les coups et permet la diffusion asymétrique.
Réseau (Client)	`GameClient.java`	Sérialisation et modèle miroir. Maintient la connexion active et actualise le modèle local.
Interface (Vue)	`NetworkObserver.java`	Découplage UI/Réseau (Pattern Observer). Notifie l'interface des changements d'état.
Réseau (Utilitaire)	`PacketParser.java`	Interprétation du protocole textuel. Assure la désérialisation des trames brutes en structures Clé/Valeur.

TABLE 13 – Architecture de la fonctionnalité Réseau

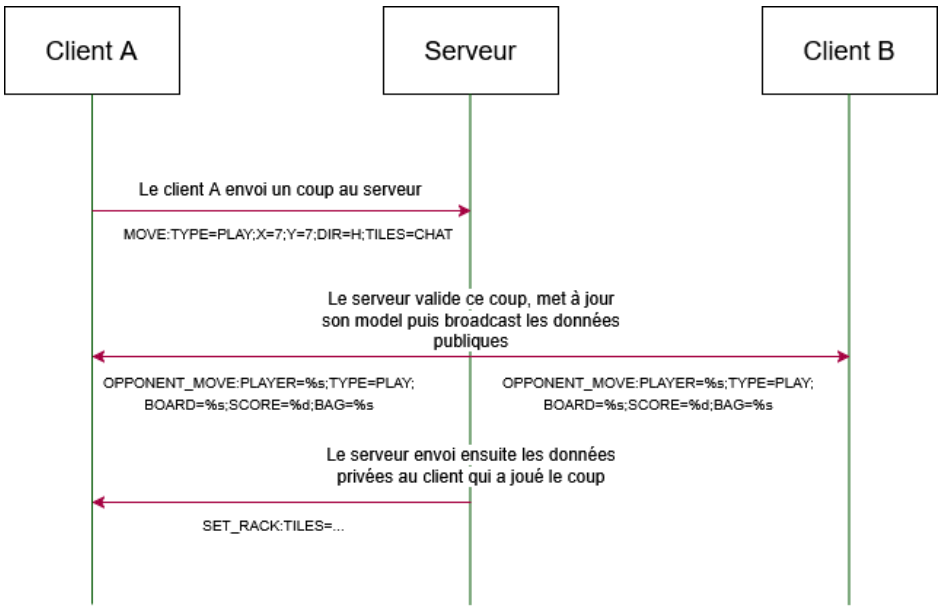


FIGURE 10 – Flux d'un coup valide dans une partie à deux clients (F39)

4.2.5 Scénario utilisateur

1. **L'utilisateur A (Hôte)** démarre le serveur de jeu depuis l'interface graphique. Le `NetworkManager` initialise le `GameServer`.

2. **L'utilisateur B** rejoint le serveur en saisissant l'adresse IP. Son `GameClient` établit la liaison avec le `ClientHandler` dédié.
3. **L'hôte** consulte la liste des joueurs connectés, sélectionne l'utilisateur B et clique sur "Inviter" (envoi d'une commande `new`).
4. **L'utilisateur B** reçoit une boîte de dialogue d'invitation et clique sur Accepter.
5. **Le serveur** instancie `OnlineGame`. Les interfaces graphiques basculent automatiquement sur la vue du plateau (`GAME_START`).
6. **L'utilisateur A** joue un coup. Le GUI transmet l'action au `NetworkManager`, qui sérialise le coup en une trame `MOVE` envoyée au serveur.
7. **Le serveur** valide le coup. Il diffuse les données publiques (`OPPONENT_MOVE`) aux deux clients, et le nouveau chevalet privé (`SET_RACK`) au joueur A.
8. **Les interfaces graphiques** reçoivent la notification via leur `NetworkObserver` et se rafraîchissent pour afficher le nouvel état global de la partie.

Stratégie de validation

Validation du scénario : Ce scénario utilisateur est automatisée et validée par le test d'intégration `testRealMultiplayerInteraction()` présent dans la classe `ClientServerTest.java` (avec un move de type PASS et sans GUI).

5 Revue Critique du Projet

L'élaboration de ce prototype de Scrabble à permis de valider la faisabilité technique d'un moteur de jeu complet couplé à des intelligences artificielles complexes. Néanmoins, avec le recul nécessaire sur le cycle de développement, plusieurs limitations architecturales, ergonomiques et algorithmiques ont été identifiées. Cette section dresse un bilan critique du projet et propose des axes d'amélioration.

5.1 Architecture Logicielle

Bien que le moteur de jeu (`core`) soit pleinement fonctionnel, son architecture actuelle présente un potentiel de refactoring important. Au fil du développement, un certain couplage s'est créé entre la logique métier (règles du jeu, gestion du plateau), les modules d'intelligence artificielle et l'interface utilisateur. Pour garantir la pérennité et la scalabilité du projet, une restructuration globale serait bénéfique :

— **Découplage et Modularité :** L'application stricte des principes de conception (comme l'injection de dépendances et le pattern MVC) permettrait de rendre les modules totalement indépendants. L'IA ne devrait interagir avec le moteur que via une API interne stricte.

— **Optimisation du MoveGenerator :** Actuellement très sollicité, le générateur de mouvements gagnerait à être optimisé (par exemple via du multithreading pour le parcours du GADDAG), afin de soulager le processeur lors des phases de réflexion de l'IA.

5.2 Ergonomie et Interface Utilisateur (GUI)

L'interface graphique, bien que stable, souffre de plusieurs lacunes ergonomiques qui altèrent la fluidité de l'expérience utilisateur (UX). Le système d'interaction avec les tuiles manque de naturel.

— **Absence de Drag-and-Drop inversé :** L'exemple le plus marquant est la gestion des erreurs de placement. Pour retirer une lettre du plateau, le joueur est contraint d'utiliser un bouton global "Annuler le placement" (*Cancel placement*). Une interaction au glisser-déposer (*drag-and-drop*) permettant de remettre directement une tuile spécifique sur le chevalet serait beaucoup plus intuitive.

— **Feedbacks visuels :** Ces rigidités d'interaction cassent le dynamisme naturel d'une partie de Scrabble physique. Des améliorations mineures, telles que des animations de placement plus fluides ou un retour visuel immédiat (surbrillance) lorsqu'un mot formé est invalide avant même sa soumission, moderniseraient considérablement l'application.

5.3 Limitations de l'Internationalisation (i18n)

À ce stade, le jeu ne supporte officiellement que deux langues : le français et l'anglais. Cette limitation n'est pas inhérente au moteur de jeu, mais plutôt à la gestion codée en dur des ressources (dictionnaires et distribution des lettres). Le système de chargement devrait être rendu totalement agnostique. L'ajout d'une nouvelle langue ne devrait nécessiter qu'un simple ajout de fichier de configuration (JSON/XML) définissant l'alphabet, la valeur des lettres, leur fréquence d'apparition et le fichier lexique associé, sans aucune recompilation du code source.

5.4 Limites des IA Implémentées

Le défi majeur de ce projet résidait dans l'implémentation de l'intelligence artificielle face au problème de l'information imparfaite. Les approches développées ont chacune révélé des limites matérielles ou logiques :

— **Approche Monte-Carlo (Minimax / Expectiminimax) :** Nos mesures empiriques ont prouvé que la complexité temporelle de ces algorithmes provoque un goulot d'étranglement majeur. L'absence de techniques d'élagage (comme l'Alpha-Beta) oblige le moteur à interrompre la recherche via un *timeout* (5 secondes) avant d'avoir pu explorer l'arbre de profondeur 2 de manière exhaustive.

— **Le Machine Learning (TensorFlow) :** L'approche offre des performances foudroyantes en temps d'inférence pour trouver des anagrammes. Cependant, l'architecture de notre tenseur d'entrée limite la vision du réseau neuronal au seul chevalet du joueur. Sans connaissance de la topologie du plateau, ce modèle est incapable de jouer de manière autonome.

5.5 Limites de l'Architecture Réseau

L'architecture client-serveur mise en place répond aux exigences du jeu en multijoueur, mais nos choix de conception présentent deux limites techniques :

— **Gestion des accès simultanés (*Thread-per-client*) :** Bien qu'elle simplifie le traitement asynchrone, l'attribution d'un thread dédié à chaque client limite le passage à l'échelle (*scalability*). Face à un volume massif de connexions simultanées, le serveur épuiserait rapidement ses ressources matérielles (CPU, mémoire), alors qu'une approche asynchrone permettrait de supporter bien plus de connexions.

— **Sécurité des échanges (Absence de chiffrement) :** Actuellement, nos trames réseau transitent en clair sur les sockets. Cette vulnérabilité expose l'application aux attaques par interception (*packet sniffing*) ou par injection de paquets modifiés. Sans couche cryptographique (de type TLS), la confidentialité et l'intégrité des communications ne sont pas garanties.

5.6 Outils d'assistance utilisés

Tout au long du projet, nous avons utilisé des assistants IA, notamment Copilot (Pro All Agents), Claude (Sonnet 4.6) et Gemini (3 Flash), comme outils d'appui pour accélérer certaines phases de développement. Leur usage a principalement concerné le débogage, la génération de cas de test et la vérification de comportements limites, tout en conservant une validation humaine systématique avant intégration dans le code final.

Références

[1] Alfred Mosher Butts. Archives de conception du jeu scrabble, 1931.

[2] Stefan Fatsis. *Word Freak*. Houghton Mifflin, 2001.

[3] Federation Internationale de Scrabble Francophone. Regles officielles du scrabble francophone, 2023. Edition 2023.

[4] Brian Sheppard et al. Quackle scrabble ai, computer scrabble project, 2006.

[5] Andrew W. Appel and Guy J. Jacobson. The world's fastest scrabble program. *Commun. ACM*, 31(5) :572–578, May 1988.

[6] Steven A. Gordon. A faster scrabble move generation algorithm. *Softw. Pract. Exper.*, 24(2) :219–232, February 1994.

[7] Robert L Harrison. Introduction to monte carlo simulation, 2010.

[8] University of Illinois Urbana-Champaign. Cs440 lecture : Games 5 - stochastic games, expectimax, 2020. Course : Artificial Intelligence (Fall 2020).

[9] VL Helen Josephine, AP Nirmala, and Vijaya Lakshmi Alluri. Impact of hidden dense layers in convolutional neural network to enhance performance of classification model, 2021.

[10] Jan Daciuk, Bruce W. Watson, Stoyan Mihov, and Richard E. Watson. Incremental construction of minimal acyclic finite-state automata. *Comput. Linguist.*, 26(1) :3–16, March 2000.

[11] Harri Hakonen Jouni Smed, Timo Kaukoranta. Aspects of networking in multiplayer computer games. *The Electronic Library*, 2002.

[12] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.

A Annexe des 40 besoins

TABLE 14: Etat de complétion des besoins F1 à F40

Besoin	Description	État
F1	Lancement depuis un exécutable ou un script du projet.	Fait
F2	Fichier de configuration `.scrabblerc` lu au démarrage, avec création et fallback.	Fait
F3	Internationalisation par défaut en anglais, avec support du français et des variables d'environnement.	Fait
F4	Interface CLI avec aide, version, verbosité, debug et codes de retour.	Fait
F5	Mode blitz avec limite de temps par joueur.	Fait
F6	Mode contest pour répondre à partir d'une position fournie.	Fait
F7	Interface graphique équivalente au CLI.	Fait
F8	Remplacement d'un joueur par une IA selon la couleur ou une valeur par défaut.	Fait
F9	Options joueurs, langue, super-scrabble, dictionnaire et liste des langues.	Fait
F10	Gestion complète des règles, des scores et de la fin de partie.	Fait
F11	Messages d'erreur utilisateur explicites sans crash.	Fait
F12	Undo/redo pour les coups humains, y compris après intervention d'une IA.	Fait
F13	Fonction de conseil pour proposer un coup.	Fait
F14	Affichage du temps blitz et arrêt immédiat quand le temps expire.	Fait
F15	Shell interactif avec les commandes demandées par la spécification.	Fait
F16	Proposition d'enregistrer la partie avant de quitter.	Fait
F17	Édition de la ligne de commande avec flèches, insertion et suppression.	Fait
F18	Historique des commandes et recherche rapide au clavier.	Fait
F19	Complétion automatique des commandes du shell.	Fait
F20	Rendu ASCII coloré et notation des coups sur plateau standard ou super.	Fait
F21	Sauvegarde et restauration ASCII d'une partie complète.	Fait
F22	Gestion des commentaires et des blocs commentés dans les sauvegardes.	Fait
F23	Format de la section `[settings]` en paires clé=valeur.	Fait
F24	Format de la section plateau avec taille variable et erreurs de ligne.	Fait
F25	Format d'historique conforme à la spécification.	Fait
F26	Chargement du lexique et choix du dictionnaire selon la langue.	Fait
F27	Interface dictionnaire abstraite et API complète.	Fait
F28	Implémentation du module DAWG.	Fait
F29	Implémentation du module GADDAG.	Fait
F30	Menus GUI File et Game conformes aux éléments imposés.	Fait
F31	Raccourcis GUI et personnalisation persistante. (Personnalisation manquante dans la partie GUI)	Partiel
F32	Drag and drop dans l'interface graphique. (On peut poser des tuiles mais pas les déplacer)	Partiel
F33	Borne de réflexion pour l'IA via `-ai-time`.	Fait
F34	Recherche des coups possibles via le GADDAG.	Fait
F35	Recherche du meilleur coup en score local maximal.	Fait
F36	Activation de l'Expectiminimax.	Fait
F37	Recherche de coups par machine learning.	Fait

Besoin	Description	État
F38	Base réseau : serveur, connexion, join, quit, ping.	Fait
F39	Serveur multi-clients avec statut, joueurs et options réseau.	Fait
F40	Invitations avancées avec acceptation, refus, annulation et timeout.	Fait